

*San Juan de la Rambla xx de xxxxx de 2023*

## **Ejercicio 01 (3,0 puntos)**

Los organizadores de un torneo de futbol le han contratado para poner los resultados de los partidos y la clasificación en Internet. El objetivo es que se creen dos ficheros **.json** de nombre **resultado\_jornada** y **clasificación**.

En el torneo participan 4 equipos que forman un grupo y juegan todos contra todos.

### **(1.0 pts) Creación del fichero resultado\_jornada.json**

En este fichero se debe guardar la información de los resultados de los partidos celebrados en la jornada. Cada día de competición se juegan 2 partidos.

La información que se debe guardar de la jornada es la siguiente:

- Fecha de la jornada
- Resultado del primer partido, la información a guardar es:
  - Nombre del equipo01
  - Nombre del equipo02
  - Goles marcados por el equipo01
  - Goles marcados por el equipo02

Esta información es solicitada al usuario y una vez obtenida se debe almacenar en una estructura de datos que considere para luego generar el fichero .json solicitado.

### **(2.0 pts) Creación del fichero clasificacion.json**

Este fichero debe guardar la información de la clasificación de los equipos después de finalizada cada jornada actualizando la clasificación de la jornada anterior.

El equipo que gana recibe 3 puntos, en caso de empate cada equipo recibe 1 punto.

La información que se debe guardar de la clasificación es la siguiente:

equipo  
nro\_partidos\_jugados  
nro\_partidos\_ganados  
nro\_partidos\_empatados  
nro\_partidos\_perdidos  
goles\_marcados  
goles\_recibidos  
puntuación del equipo

Evidentemente al ser una clasificación, ésta debe estar ordenada por el campo de la puntuación de mayor a menor.

En caso de que haya empate entre los equipos se debe decidir quien va primero siguiendo las siguientes reglas:

- Mayor diferencia de goles (que se calcula como la resta de los goles marcados menos los recibidos).
- Si en el punto anterior hay empate, entonces irá primero el que más goles haya marcado.
- Si persiste el empate, se decide quien va primero al generar un número aleatorio entre 1 y 10. Si sale entre 1 y 5 irá un equipo y sino el otro.

Como se puede apreciar la clasificación es acumulativa, es decir, la puntuación es la de todas las jornadas jugadas hasta el momento. Por tanto, para crear el fichero de la siguiente jornada previamente se habrá tenido que leer el fichero de la anterior, para cargar la puntuación hasta la jornada anterior y luego actualizar los datos con los resultados con los resultados de la nueva.

Esto quiere decir que aquí se deberá leer también el fichero .json generado en el apartado anterior. Tener en cuenta que cuando se vaya a procesar la primera jornada, no hay fichero de clasificación. Lo que se le pide:

Un programa en Python que genere los dos ficheros .json solicitados.

### Ejercicio 01 (3,0 puntos)

Una empresa dedicada al comercio al por mayor de frutas y verduras envía al contable los datos de su inventario mediante un fichero .csv con las siguientes características.

```
codigo;descripcion;embalaje;peso;precio;proveedor
v0001;tomate pera;paquete;0,5;2,45;mercaterife
f0001;manzanas golden;caja;10;17,35;frutisimo
v0002;cebolla;bolsa;2,0;1,80;agricultoria
f0002;pera conferencia;caja;8;14,85;mercaterife
```

En un proceso de expansión la empresa compra otro comercio que se dedica al mismo ramo, a la hora de analizar el fichero de datos que tiene para llevar su inventario se ve que es un fichero .csv con las siguientes características.

```
cod001;verdura;mercaterife;tomate pera;paquete;17;2.35
cod002;fruta;bestfruit;manzanas golden;caja;12;17.35
cod003;verdura;agricultoria;ajos;bolsa;50;1.80
cod004;fruta;frutisimo;pera conferencia;caja;7;14.85
cod005;fruta;mercaterife;pera devoer;caja;8;14.85
cod006;verdura;comarca_sjr;papas;bolsa;10;18.95
```

El contable sugiere que se deben uniformizar ambos inventarios en un único fichero y propone que éste nuevo fichero de inventario se realice en formato .json.

Para la creación del nuevo fichero .json se propone la siguiente estructura de datos:

```
inventario = [ { 'codigo':valor_codigo, 'descripcion':valor_descripcion,  
                'embalaje':valor_embalaje, 'peso':valor_peso,  
                'precio':valor_precio, 'proveedores':[valor_proveedores] }  
                ... {...}.... {} ]
```

Una lista de diccionarios. Ese decir, una lista donde cada elemento es un diccionario

Para llenar esta estructura se debe tener en cuenta lo siguiente:

- Los datos salen de los dos ficheros .csv anteriores. Resaltar que uno de los ficheros tiene encabezado y el otro no.
- PRIMERO SE DEBE PROCESAR el fichero .csv que tiene la empresa y luego el de la empresa que se compró. Esto se hace así para mantener los códigos propios ya que éstos son distintos.
- Dos productos se consideran iguales si COINCIDEN EN SU DESCRIPCIÓN. Por ejemplo, si los dos dicen “tomate pera”.
- En el caso de que dos productos sean iguales, se deben sumar las cantidades que aparecen en cada inventario y mantener el mayor de los precios, además, se debe almacenar uno o los dos proveedores (si son distintos) en la lista propuesta para ello.
- Si al procesar el segundo fichero .csv aparecen productos que no hay en el primer fichero, se debe crear una nuevo código para registrarlo. Los últimos códigos usados por la empresa han sido los siguientes:
  - Para la verdura ‘v0002’
  - Para la fruta ‘f0002’

El nuevo código que se genere se debe añadir 1 a la numeración del último.

Se pide un programa en Python que leyendo los dos ficheros .csv que se les entregan, creen la nueva estructura de datos que se sugiere y genere el fichero .json

El fichero .json se debe llamar “inventario.json”

El fichero .csv de la empresa se llama “empresa.csv”

El fichero .csv de la empresa que se compra se llama “comprada.csv”

El diccionario de la estructura a crear se llama inventario.

Puntaje:

Lectura y proceso del primer fichero csv 0,75 pts

Lectura y proceso del segundo fichero csv 1,50 pts

Generación del fichero .json 0,75 pts.

## **Ejercicio 02 (4,0 punto)**

Una empresa de comidas fast-food quiere implementar un programa para llevar información de sus platos y los ingredientes que estos contienen.

La estructura de datos que pretender utilizar es la siguiente:

Un diccionario con tres claves:

```
comidas = { 'entrantes' : [ { 'plato' : nombre, 'ingredientes':[ ingr01, ...ingr0n]} ,  
                             { 'plato' : nombre, 'ingredientes':[ ingr01, ...ingr0n]}  
                             ... ],  
            'pizzas' : [ { 'plato' : nombre, 'ingredientes':[ ingr01, ...ingr0n]} ,  
                          { 'plato' : nombre , 'ingredientes':[ ingr01, ...ingr0n]}  
                          ... ],  
            'pastas' : [ { 'plato' : nombre, 'ingredientes':[ ingr01, ...ingr0n]} ,  
                          { 'plato' : nombre , 'ingredientes':[ ingr01, ...ingr0n]}  
                          ... ] }
```

El valor de cada clave es una lista que contiene diccionarios con los platos que ofrece el restaurante. Estos diccionarios tienen dos claves: plato e ingredientes donde ingredientes es una lista.

La empresa quiere tener un menú como el siguiente:

1. Cargar fichero .pickle
2. Añadir plato.
3. Buscar platos por ingredientes
4. Eliminar plato.
5. Escribir fichero .pickle
6. Salir

Lo que hace cada una de las opciones

#### Opción 1. Cargar fichero .pickle (0,25 pts)

Esta opción abre el fichero .pickle si existe y carga los datos de las comidas que ofrece el establecimiento. Si no existe crea la estructura comidas vacía.

#### Opción 2. Añadir plato (0,50 pts)

Esta opción se ejecuta si el diccionario **comidas** tiene elementos, en caso contrario se debe indicar un mensaje de error.

Esta opción pide el nombre del plato a añadir y la categoría a la que pertenece. Si el plato ya existe se indica mensaje de error.

Si no existe, se pide mediante un bucle todos los ingredientes que tiene. Para salirse del bucle se debe presionar la tecla <INTRO>. Un plato debe tener al menos 1 ingrediente.

#### Opción 3. Buscar plato por ingrediente (2,50 pts)

Esta opción se ejecuta si el diccionario **comidas** tiene elementos, en caso contrario se debe indicar un mensaje de error.

Esta opción comienza mediante un bucle a pedir los ingredientes a buscar al usuario. De este bucle se sale si se han introducido hasta 4 ingredientes o se utiliza la tecla <INTRO> para salir. NO SE PUEDE PEDIR LA CATEGORÍA DEL PLATO.

Luego se busca en la estructura de datos todos los platos que contengan la totalidad de ingredientes solicitados.

Si hay platos que cumplan con la búsqueda se deben mostrar su categoría, su nombre y todos los ingredientes que contiene. En caso de que no haya se debe mostrar la frase “NO hay platos que contengan todos los ingredientes solicitados”.

#### Opción 4. Eliminar plato (0,50 pts)

Esta opción se ejecuta si el diccionario **comidas** tiene elementos, en caso contrario se debe indicar un mensaje de error.

Esta opción pide el nombre del plato a eliminar y la categoría a la que pertenece, si no existe se indica mensaje de error. Si existe, se elimina de la estructura

Opción 5. Escribir fichero .pickle (0,25 pts)

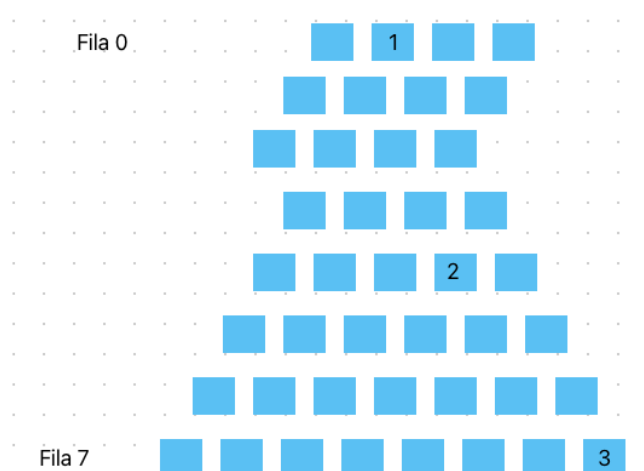
Esta opción escribe el fichero .pickle con la estructura de datos del programa siempre y cuando la estructura NO ESTÉ VACÍA.

Escriba un programa de Python que permita gestionar los platos de comida de la empresa. Cada opción del menú debe ejecutar una función que permita resolver los solicitado.

Ejercicio 01 (4,0 pts):

Una empresa dedicada a la proyección de películas de cine le contrata para llevar realizar un algoritmo que le permita llevar el control de la ocupación de una sala de sus salas.

La sala a controlar tiene la forma que se muestra a continuación:



La sala tiene 8 filas, en cada fila se tiene la cantidad de asientos que se muestran en la figura.

Las filas se identifican con los valores de f0 hasta f7

En cada fila, las butacas van de la número 1 comenzando por la izquierda.

Como ejemplo:

la butaca etiquetada con 1 en la figura se identifica como f0-2

la butaca etiquetada con 2 en la figura se identifica como f4-4

la butaca etiquetada con 3 en la figura se identifica como f7-8

Para indicar que una butaca está libre se indica con un “-“, si está ocupada se indicará con “X”

Deberá pensar cual será la estructura de datos que utilizará para almacenar las butacas de la sala de cine. Recuerde que inicialmente todas las butacas están libres. (Escriba en el encabezado de su programa como comentario una breve reseña de la estructura de datos elegida)

Lo que se debe hacer parte01

7. **(0,5 pts)** Escriba la función **creaSalaButacas()** que crea la estructura de datos y almacena la distribución de butacas mostrada en la figura anterior. Recuerden que inicialmente las butacas están todas libres. Esta función debe retornar el tipo de estructura que usted ha definido.

Una vez creada la estructura de datos el programa debe mostrar un **prompt** para poder operar con él. En el PROMPT se indican las ordenes para realizar operaciones de consulta de butacas libres y venta de entradas para una función.

El prompt a mostrar será el siguiente **“SALA\_CINE => “**

Cuando el programa muestra el prompt se queda a la espera a que se introduzcan las ordenes que se quieren realizar.

Las ordenes a desarrollar serán las siguientes:

**MOSTRAR** que muestra el estado actual de ocupación de la sala.

**CONSULTAR nro\_butacas\_buscar fila\_inicial fila\_final:** realiza la consulta en la estructura para ubicar nro\_entradas libres consecutivas en una fila. La búsqueda se realiza desde la fila\_inicial hasta la fila\_final ambas incluidas.



**VENDER** **nro\_entrada id\_butaca01, id\_butaca02 ,... id\_butaca0n**: realiza la venta de las butacas que tienen los identificadores que se pasan por parámetros. Más adelante se explica el formato de ese parámetro.

**SALIR**: termina la ejecución del programa

Lo que se debe hacer parte02:

(0,5 PTS) Crear la función **mostrarButacas()** que muestra por pantalla de forma tabulada y bien organizada el estado actual de ocupación de la sala “NO SE PERMITE LA ESCRITURA DE FORMA DIRECTA DE LA ESTRUCTURA DE DATOS, SE DEBE RECORRER PARA ESCRIBIR LOS DATOS BUTACA A BUTACA”

(1,5 PTS) Crear la función **consultarButacasLibres()** que recibe como parámetros los siguientes valores:

**nro\_butacas\_buscar**: es el número de butacas libres que se necesitan

**fila\_inicial**: la fila desde donde se quiere comenzar la búsqueda ( opcional ). El valor por defecto es 0 correspondiente a la fila 0

**fila\_final**: la fila hasta donde se permite la búsqueda de butacas libres (opcional ). El valor por defecto es 7 correspondiente a la fila 7 de la sala.

Esta función debe buscar en TODA LA SALA, donde hay **nro\_butacas\_buscar** LIBRES CONSECUTIVAS en la misma fila. La función retorna una copia de la estructura de datos que almacena las butacas en donde se indicará con “@” aquellas butacas que están libres consecutivas. Es decir se cambiarán los ‘-’ por ‘@’ donde se cumpla la condición. Tenga en cuenta que el cambio es sólo para su muestra por pantalla y no afecta a la estructura principal.

(1,5 PTS) Crear la función de nombre **ocuparButacas()** que recibe como parámetros los siguientes valores:

**nro\_butacas**: el número de butacas que se deben marcar como ocupadas

**butacas\_libres**: uno o más identificadores de butacas que son las que se van a ocupar.

Este último parámetro puede utilizar el identificador de cada butaca, o se puede abreviar de la forma que se explica en el siguiente ejemplo:

Si se compran 4 entradas y las butacas que desea el cliente son las butacas : f6-3, f6-4, f6-5 y f6-6

La función ocupar\_butacas() podrá llamarse de las dos formas siguientes:

**ocuparButacas(4, f6-3, f6-4, f6-5 y f6-6)** o de esta forma más abreviada **ocuparButacas(4, f6-3-6)** que indica las butacas de la 3 a la 6 de la fila 6.

Realice un programa en Python con todas las funciones solicitadas y un programa principal que permita procesar las distintas opciones del prompt.

Ejercicio 02 (3,0 pts):

Diseñe una función **recursiva** que recibe como parámetro una lista de números y letras y retorne dos listas ordenadas, una con todos los números y la otra con todas las letras.

La lista puede tener solamente un nivel de profundidad, esto quiere decir que dentro de la lista principal pueda existir otra lista con números y letras pero no debe tener un segundo nivel de lista:

Veamos un ejemplo:

Lista válida [ 2, 'a', [1, 'b'] 'd', 'c', 3, ['z', 'a', 4], 5]

Lista NO valida [ 2, 'a', [1, b] 'd', 'c', 3, ['z', **[7]**, 'a', 4], 5]

El programa principal debe ejecutar la función recursiva e imprimir las dos listas ordenadas generadas.

Para el ejemplo válido puesto arriba la solución debe ser:

[1,2,3,4,5] y ['a', 'a', 'b', 'c', 'd', 'z']

### Ejercicio 03 (3,0 pts):

Se pide diseñar dos funciones para trabajar con listas de números y series

(1,5 pts.) La primera función **encuentraNroSeries( lista )**

Diseña la función `encuentraNroSeries` que recibe una lista como parámetro y retorna el número de series que se encuentra en la lista.

Se llama serie a todo tramo de la lista con valores idénticos. La lista es de números enteros.

Por ejemplo, la lista `[1, 1, 8, 8, 8, 8, 0, 0, 0, 2, 10, 10]` tiene 5 series (ten en cuenta que el 2 forma parte de una serie de un solo elemento).

La segunda función `posicionSerieMasLarga( lista )`

La función `posicionSerieMasLarga` recibe como parámetro una lista de números enteros y debe retornar el índice/los índices (posición/posiciones) donde comienza la serie más larga.

En el ejemplo del ejercicio anterior, la «serie» más larga empieza en la posición 2 (que es el índice donde aparece el primer 8).

Si hay dos o más «series» de igual longitud y ésta es la mayor, debes devolver las posiciones donde comienza cada una de las series.

Por ejemplo, para `[8, 2, 2, 9, 9]` deberás devolver la posición 1, 3.

### Pregunta 01 (3 puntos)

Se pide diseñar un programa de un juego que permita adivinar un número generado aleatoriamente en un determinado número de jugadas.

El juego comienza mostrando un prompt como el siguiente:

`( n , m ) ->`

Donde n y m inicialmente tendrán el valor ( '\*', '@' ) ya que no se ha generado el número y tampoco el nro de intentos que se dispone para adivinarlo.

## Comando "COMENZAR"

( '\*', '@' ) -> COMENZAR | comenzar | Comenzar....

Indicará al programa que se debe generar un número entero aleatorio entre 1.000 y 999.999 (mil y un millón menos 1).

Una vez generado el número a adivinar, se procede a calcular el número de intentos que se dispone para averiguarlo. En este caso, ese valor se determina por el número de dígitos que tiene el número que se debe adivinar.

Una vez hallados los dos valores se muestra al usuario un string con tantos "%" como números tenga.

Además se debe actualiza el prompt con los siguientes datos:

El '\*' debe sustituirse por el nro de dígitos distintos que faltan por adivinar (este valor se debe calcular mediante una función que usted debe diseñar.

El '@' se debe sustituir por el número de intentos que aun quedan por jugar.

Esto se entiende mejor con un ejemplo:

( '\*', '@' ) -> COMENZAR

Y el ordenador genera el número **124561**, y por tanto el número de intentos que dispone el jugador para adivinarlo es de 6.

Si nos fijamos en el número generado, a pesar de tener 6 dígitos, sólo tiene 5 distintos, a saber: 1, 2, 4, 5, 6. Por tanto el valor a poner en el prompt sería 5.

El nuevo prompt debe quedar de la siguiente forma

( '\*', '@' ) -> COMENZAR (el prompt anterior)

Número a adivinar: %%%%%%%%%%

( 5, 6 ) ->

## Comando "JUGAR"

Siguiendo el ejemplo nos disponemos a jugar

( 5, 6 ) -> JUGAR | jugar | Jugar.... dígito\_a\_jugar

Con este comando se indica el dígito que queremos jugar para ver si está dentro del número.

Esta opción se debe realizar con una función que recibe como parámetros el número a adivinar y el patrón de porcentajes con los dígitos que se han ido averiguando. La función retorna el patrón de aciertos actualizado.

Dependiendo de si el dígito esté o no, se debe actualizar el prompt, ya que se ha gastado un turno y si se ha acertado, se debe actualizar el número de cifras distintas pendientes de adivinar.

Tener en cuenta que si el dígito jugado está más de una vez, se debe cambiar todas las ocurrencias en las que aparezca.

Esta opción genera una advertencia si no hay número que adivinar.

### Ejemplos

( 5, 6 ) -> JUGAR 2

Número a adivinar: %2%%%%

Si ahora jugamos el 1

(4, 5) -> JUGAR 1

Número a adivinar: 12%%%%1

(3, 4) -> JUGAR 9

Número a adivinar: 12%%%1

(3, 3) -> JUGAR 9

Y así sucesivamente hasta llegar al fin del juego.

¿Cómo se termina el juego? QUE NO EL PROGRAMA

Dos formas....

### **1. El usuario adivina el número antes de que se acaben los intentos**

Se indica que el jugador ha ganado. En este caso, se muestra el número y se indica el número de intentos en el que ha logrado..

### **2. Se acaban los intentos y el usuario no ha adivinado el número.**

Se indica que el jugador ha perdido y muestra el número que no adivinó.

En ambos casos el programa muestra el siguiente prompt una vez escritos los mensajes correspondientes.

( '\*', '@' ) ->

## **Comando “SALIR”**

( '\*', '@' ) -> SALIR | salir | Salir

Esta opción termina el programa.

## **Ejercicio 02 (4,0 pts):**

Una empresa de automatización de la compra de comestibles, te acaba de contratar para trabajar en su aplicación de venta e inventario.

La empresa quiere mantener el inventario actualizado cada vez que se procese una venta en uno de los dos tipos de TPV con los que trabaja.

En cada TPV se procesa la venta y se genera un fichero que varía según el tipo de TPV. Este fichero es el que se utilizará para actualizar el inventario.

### ***Estructuras de datos a actualizar***

La empresa quiere llevar su inventario en un diccionario que también quiere serializar mediante el uso de Pickle.

El diccionario de nombre **Inventario** será de la siguiente forma:

Inventario = { codigo\_01: cantidad\_01, ... , codigo\_0n: cantidad\_0n }

donde:

**código**: cualquiera de los códigos de los productos de los que se dispone en la tienda

**cantidad**: es la cantidad actual que hay de ese producto en el almacén.

El fichero donde se guardará la serialización del inventario tendrá el nombre **inventario.pckl**.

### ***Características de los TPV***

Las ventas de productos se realizan a través de los TPV, la empresa tiene dos modelos, que se detallan a continuación:

TPV-csv

La característica de este TPV es que genera un fichero **csv sin cabecera** donde se almacenan los datos de la venta realizada. Este fichero es el que se debe utilizar para actualizar el inventario. El nombre del fichero es **venta.csv**.

TPV-json

La característica de este TPV es que genera un fichero **json** donde se almacenan los datos de la venta realizada. Este fichero es el que se debe utilizar para actualizar el inventario. El nombre del fichero es **venta.json**.



Para la generación de ambos ficheros la información que se debe almacenar es la que se muestra a continuación:

fecha: la fecha en la que se realiza la venta

hora: la hora de la venta

El cod\_producto y cantidad que se vende de cada producto adquirido por el cliente

### ***El programa***

El programa a realizar tendrá el siguiente menú:

1. Cargar datos inventario
2. Grabar datos inventario
3. Venta mediante TPV-csv
4. Venta mediante TPV-json
5. Actualizar inventario desde TPV-csv
6. Actualizar inventario desde TPV-json
7. Salir del programa

Explicación de lo que hace cada opción.

#### ***1. (0,50 puntos) Cargar datos inventario***

Esta opción carga los datos del fichero **inventario.pckl** al programa (se debe realizar cada vez que se comience a ejecutar el programa). Se puede asumir que existe un fichero .pckl vacío al comienzo del programa.

#### ***2. (0,50 puntos) Grabar datos inventario***

Esta opción graba el diccionario inventario en el fichero **inventario.pckl**. Esta opción deberá ejecutarse cada vez que se actualice el inventario o antes de salir del programa.

#### ***3. (0,75 puntos) Venta mediante TPV-csv***

Pide al usuario los siguientes datos:

Día de la operación:

Hora de la operación:

Los productos que ha comprado y en que cantidad. La compra termina al poner '@' en el código de producto.

Con los datos obtenidos de la lectura se genera un fichero .csv sin encabezado de nombre **venta.csv**.

Tienen que tener en cuenta la estructura de datos que deben utilizar para generar el fichero solicitado.

#### **4. (0,75 puntos) Venta mediante TPV-json**

Pide al usuario los siguientes datos:

Día de la operación:

Hora de la operación:

Los productos que ha comprado y en que cantidad. La compra termina al poner '@' en el código de producto.

Con los datos obtenidos de la lectura se genera un fichero .json de nombre **venta.json**.

Tienen que tener en cuenta la estructura de datos que deben utilizar para generar el fichero solicitado.

#### **5. (0,75 puntos) Actualizar inventario desde TPV-csv**

En esta opción se lee el fichero **venta.csv** y con los datos obtenidos se actualiza el inventario.

#### **6. (0,75 puntos) Actualizar inventario desde TPV-json**

En esta opción se lee el fichero **venta.json** y con los datos obtenidos se actualiza el inventario.

Ejercicio 03 (3,0 pts):

Se quiere crear una estructura de datos con la información de los movimientos de la caja de una tienda de ropa. Como es sabido en la tienda se puede pagar en

efectivo o mediante tarjeta y además de la compra también se puede realizar la devolución de los artículos.

La estructura de datos a crear es una lista donde cada elemento será un diccionario. Esta lista recibe el com de **ventas**.

ventas = [ venta01, venta02, ... , venta0n ], donde venta01 ... venta0n son diccionarios.

Cada diccionario venta0x será de la siguiente forma:

La clave es: (fecha\_compra, hora\_compra)

El valor es: una lista que puede contener distintos campos según la operación a guardar.

(fecha\_compra, hora\_compra): [ distintos campos según la operación realizada ]

Las operaciones permitidas y la información a guardar es la siguiente:

- Si la operación es una compra en efectivo:

Se debe guardar : [ “caja”, “efectivo”, nro\_ticket\_compra, importe, dinero\_recibido]

- Si la operación es una compra pagada con tarjeta:

Se debe guardar : [ “caja”, “tarjeta”, nro\_ticket\_compra, importe, dinero\_recibido, banco, nro\_tarjeta]

- Si la operación es una devolución en efectivo:

Se debe guardar : Se debe guardar : [ “dev”, “efectivo”, nro\_ticket\_compra, fecha\_dev, importe]

- Si la operación es una devolución por tarjeta:

Se debe guardar : [ “dev”, “tarjeta”, nro\_ticket\_compra, fecha\_dev, importe, banco, nro\_tarjeta]

**Parte01 (2,0 pts) (opciones 1 y 2 del menú)**

Desarrolle un programa en Python que tenga un menú de dos opciones.

La primera opción es la que permite crear la estructura de datos solicitada, y pida datos al usuario para llenarla. En el programa se debe contemplar todas las posibles operaciones que se pueden realizar en la tienda.

La segunda opción es la de mostrar línea a línea el contenido de la estructura de datos generada en la opción anterior.

## **Parte02: (1,0 pts) (la opción 3 del menú)**

Esta opción consiste en recorrer la estructura de datos creada en el apartado 1 y mostrar por pantalla los siguientes datos:

Ventas:

Número ventas pagadas en efectivo

Importe de las ventas pagadas en efectivo

Número ventas pagadas con tarjeta

Importe de las ventas pagadas en tarjeta

Devoluciones:

Número devoluciones realizadas en efectivo

Importe de las devoluciones realizadas en efectivo

Número devoluciones realizadas con tarjeta

Importe de las devoluciones realizadas con tarjeta.

Cada una de las opciones del menú se desarrollan mediante una función.

### **Ejercicio 01 (3,0 pts):**

Escribir un programa en Python que utilice una función recursiva que cuente las vocales de cada tipo hay en un string que se le pide al usuario por pantalla y el total de vocales que se encuentran en la frase.

La función recursiva de nombre **clasificaVocales()** recibe como parámetro un string y retorna una lista de 6 elementos. Cada uno de los elementos de esa lista retorna es un número que indica el numero de vocales por tipo y total encontradas en la frase.

Si la frase que se introduce es:

**El melocotón está maduro y listo para comer.**

La lista que se debe retornar debe ser la siguiente:

[ 4, 4, 1, 6, 1, 16 ]

Cada elemento corresponde a:

Primer elemento = El número de vocales 'a', 'A', 'á', 'Á' que hay en el string.

Segundo elemento = El número de vocales 'e', 'E', 'é', 'É' que hay en el string.

Tercer elemento = El número de vocales 'i', 'I', 'í', 'Í' que hay en el string.

Cuarto elemento = El número de vocales 'o', 'O', 'ó', 'Ó' que hay en el string.

Quinto elemento = El número de vocales 'u', 'U', 'ú', 'Ú' que hay en el string.

Sexto elemento = El número total de vocales que hay en el string.

Ejercicio 02 (4,0 pts):

Queremos crear un programa que trabaje con fracciones **a/b**. Para representar una fracción vamos a utilizar dos enteros: numerador y denominador.

Vamos a crear las siguientes funciones para trabajar con funciones:

- **Leer\_fracción:** La tarea de esta función es leer por teclado el numerador y el denominador. Cuando leas una fracción debes simplificarla. ( **0,25 pts.** )
- **Escribir\_fracción:** Esta función escribe en pantalla la fracción. Si el denominador es 1, se muestra sólo el numerador. ( **0,25 pts.** )
- **Calcular\_mcd:** Esta función recibe dos números enteros y devuelve el máximo común divisor de los dos. ( **0,50 pts.** )

## El algoritmo

El algoritmo de Euclides para encontrar  $MCD(A,B)$  es como sigue:

- Si  $A = 0$  entonces  $MCD(A,B)=B$ , ya que el  $MCD(0,B)=B$ , y podemos detenernos.
- Si  $B = 0$  entonces  $MCD(A,B)=A$ , ya que el  $MCD(A,0)=A$ , y podemos detenernos.
- Escribe  $A$  en la forma cociente y residuo ( $A = B \cdot Q + R$ ).
- Encuentra  $MCD(B,R)$  al usar el algoritmo de Euclides, ya que  $MCD(A,B) = MCD(B,R)$ .

- **Simplificar\_fracción:** Esta función simplifica la fracción, para ello hay que dividir numerador y denominador por el MCD del numerador y denominador. ( 0,50 pts. )
- **Sumar\_fracciones:** Función que recibe dos funciones  $n1/d1$  y  $n2/d2$ , y calcula la suma de las dos fracciones. La suma de dos fracciones es otra fracción cuyo numerador= $n1*d2+d1*n2$  y denominador= $d1*d2$ . Se debe simplificar la fracción resultado. ( 0,50 pts. )
- **Restar\_fracciones:** Función que resta dos fracciones: numerador= $n1*d2-d1*n2$  y denominador= $d1*d2$ . Se debe simplificar la fracción resultado. ( 0,50 pts. )
- **Multiplicar\_fracciones:** Función que recibe dos fracciones y calcula el producto, para ello numerador= $n1*n2$  y denominador= $d1*d2$ . Se debe simplificar la fracción resultado. ( 0,50 pts. )
- **Dividir\_fracciones:** Función que recibe dos fracciones y calcula el cociente, para ello numerador= $n1*d2$  y denominador= $d1*n2$ . Se debe simplificar la fracción resultado. ( 0,50 pts. )

Crear un programa principal que utilizando las funciones anteriores muestre el siguiente menú: ( 0,50 pts. )

1. Sumar dos fracciones:
2. Restar dos fracciones:
3. Multiplicar dos fracciones:
4. Dividir dos fracciones:
5. Salir

Las opciones realizan lo siguiente:

1. En esta opción se piden dos fracciones y se muestra el resultado de la suma.
2. En esta opción se piden dos fracciones y se muestra el resultado de la resta.
3. En esta opción se piden dos fracciones y se muestra el resultado del producto.
4. En esta opción se piden dos fracciones y se muestra el resultado de la división.

Ejercicio 03 (3,0 pts):

Escribe una función que reciba dos palabras (String) y retorne verdadero si las dos palabras son un anagrama o falso en caso contrario.

Un anagrama es una palabra que se forma a partir de otra reordenando todas sus letras sin ser la misma.

Ejemplos de anagramas.

Tengan en cuenta las vocales acentuadas, las letras mayúsculas y minúsculas.

- Cara - arca.
- Lácteo - coleta.
- Retener - enterré
- Castor - castro.
- Seto - esto.
- Nido - Odín.
- Lobo - bolo.